

VEHICLE RENTAL SYSTEM

Database Management System Project Report

Implementation Phase: Design, Database and Java Desktop Application

Submitted by

Arush Chaudhary - 25BCE0754

Prachurjya P. Kalita - 25BCE2337

Sangram Umesh Waghade - 25BCE0737

Submitted to: Dr. Pooja Chaudhary

Course: Database Management Systems (DBMS)

Academic Year: 2026–2027

Abstract

The Vehicle Rental System is a database application for renting cars, bikes and scooters. Customer, employee, vehicle, category, pricing, booking and payment data are stored in a MySQL database, and a Java Swing desktop application connected through JDBC gives customers and employees their own interfaces. The central rule is that a vehicle can never have two overlapping active bookings; it is enforced inside MySQL by triggers that lock the vehicle row, so it holds even under concurrent requests.

Keywords: DBMS, MySQL, JDBC, Java Swing, ER Diagram, Normalisation, Triggers, Transactions, Referential Integrity.

1. INTRODUCTION

1.1 Problem Statement

A rental business handles many vehicles, customers, rental periods and prices. Without a central database it is hard to know which vehicles are free, to spot conflicting reservations and to calculate charges consistently. The key problem is preventing the same vehicle from being booked by two customers for overlapping periods, even when both request it at the same moment, while keeping what customers may do separate from what staff may do.

1.2 Objectives

- Design a relational database normalised to Third Normal Form (3NF).
- Prevent overlapping active bookings for the same vehicle, including under concurrent requests.
- Calculate charges from category- and duration-based pricing rules and keep payment and refund records.
- Build a Java desktop application over JDBC with secure, role-based access for customers and employees.
- Record every SQL statement executed by the application in a query log.

1.3 Scope and Roles

Customers sign up, search available vehicles, request bookings, pay after an employee confirms, and cancel before pick-up. **Employees** (Manager or Rental Operator) confirm, add, update, cancel and delete bookings (CRUD), view payments, add vehicles or mark them under maintenance, see a dashboard of monthly sales by vehicle type, the query log and an interactive ER diagram. Only a Manager can add staff. A real payment gateway, GPS tracking and insurance claims are outside the scope.

2. REQUIREMENT ANALYSIS

2.1 Functional Requirements

ID	Requirement	Description
FR-01	Registration and login	Customers sign up; customers and employees log in with salted SHA-256 password hashes.
FR-02	Role-based access	Separate screens for customers and employees; only managers manage staff.
FR-03	Fleet and pricing	Add and edit vehicles, set status, keep daily rates and discounts by category and duration.
FR-04	Availability search	List vehicles free for a period, filtered by type, fuel and transmission.

ID	Requirement	Description
FR-05	Booking (CRUD)	Customers request (Pending); employees create, confirm, update, complete, cancel, delete.
FR-06	Conflict prevention	No overlapping non-cancelled bookings for the same vehicle.
FR-07	Payments	Pay only after confirmation; refunds recorded automatically on cancellation.
FR-08	Dashboard, log, ER viewer	Monthly sales by vehicle type, query log of every SQL statement, live ER diagram.

2.2 Key Business Rules

- Pending, Confirmed and Completed bookings block a vehicle; Cancelled bookings do not.
- A vehicle under maintenance or retired cannot be booked.
- The agreed rate, discount and total are stored with the booking, so later price changes never alter old bills.
- Successful payments may never exceed the booking total; bookings with payments are cancelled, never deleted.

2.3 Overlap Rule and Pricing

Rental periods are half-open intervals [StartDate, EndDate): a rental ending on 15 November leaves the vehicle free on 15 November. A new booking overlaps an existing one if:

ExistingStart < NewEnd AND ExistingEnd > NewStart

TotalAmount = Duration × AgreedDailyRate × (1 − DiscountPercent / 100)

Example: a car for 4 days falls in the 3–6 day tier, so 4 × ₹1,300 × 0.95 = ₹4,940.

2.4 Booking Life Cycle

Status	Set by	Meaning	Next allowed status
Pending	Customer (request) or employee	Requested; the vehicle is held for these dates	Confirmed, Cancelled
Confirmed	Employee	Approved; the customer can now pay	Completed, Cancelled
Completed	Employee	Vehicle returned	—
Cancelled	Customer (before pick-up) or employee	No longer blocks the vehicle; any payment is refunded	—

3. ER DIAGRAM DESIGN

The ER model uses Chen notation: rectangles for entities, ellipses for attributes (keys underlined) and diamonds for relationships. Compared with the analysis phase it adds the Employee entity, the HANDLES relationship and the password and vehicle-detail attributes.

Entity	Key attributes	Purpose
Customer	CustomerID, Email, Phone, DrivingLicenceNo, password salt and hash	Customer identity, contact details and online login
Employee	EmployeeID, Email, Designation, password salt and hash	Staff: Manager or Rental Operator
VehicleCategory	CategoryID, CategoryName	Car, Bike, Scooter
Vehicle	VehicleID, RegistrationNo, Make, Model, OperationalStatus, BodyType, FuelType	A rentable vehicle

Entity	Key attributes	Purpose
PricingRule	PricingRuleID, MinDays, MaxDays, DailyRate, DiscountPercent, EffectiveFrom	Rates by category, duration and date
Booking	BookingID, StartDate, EndDate, BookingStatus, AgreedDailyRate, TotalAmount	The rental with its agreed price
Payment	PaymentID, Amount, PaymentMethod, PaymentStatus, TransactionRef	Payments and refunds of a booking

Relationship	Entities	Cardinality	Meaning
MAKES	Customer — Booking	1 : N	A customer can make many bookings.
ASSIGNED TO	Booking — Vehicle	N : 1	A vehicle has many bookings over time, without overlaps.
HAS	Booking — Payment	1 : N	A booking can have several payments and refunds.
CLASSIFIES	VehicleCategory — Vehicle	1 : N	Each vehicle belongs to one category.
DEFINES RATE	VehicleCategory — PricingRule	1 : N	A category has several pricing tiers.
APPLIES TO	PricingRule — Booking	1 : N	A rule is applied to many bookings.
HANDLES	Employee — Booking	1 : N	An employee handles many bookings (partial: a pending booking has none).

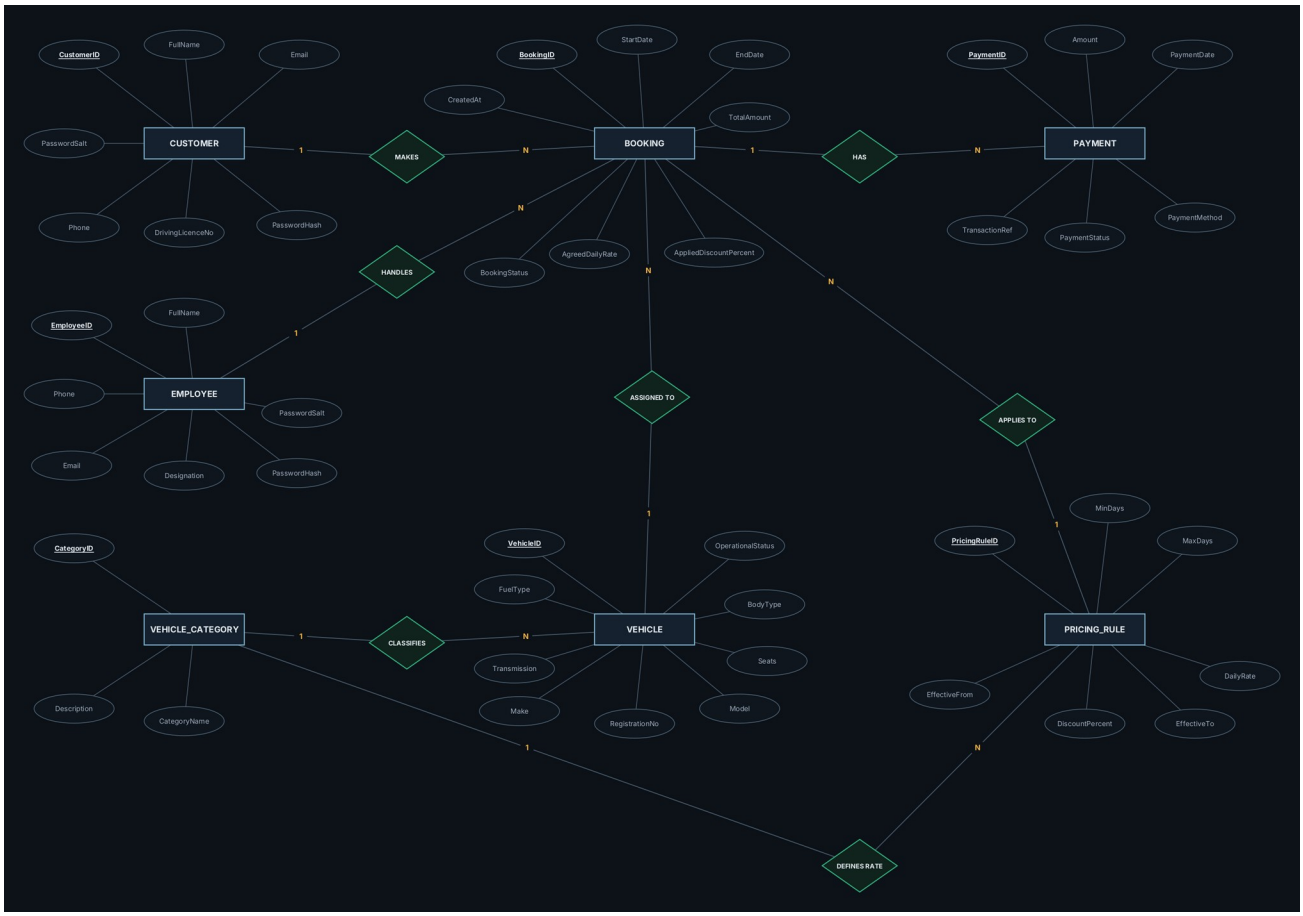


Figure 3.1: ER diagram of the Vehicle Rental System (Chen notation)

4. RELATIONAL MODEL

Each entity becomes a table, and every one-to-many relationship is mapped by placing the foreign key on the “many” side. QUERY_LOG is added for auditing.

Relation	Attributes (PK underlined in the diagram; FK marked)
CUSTOMER	CustomerID (PK), FullName, Email, Phone, DrivingLicenceNo, PasswordSalt, PasswordHash
EMPLOYEE	EmployeeID (PK), FullName, Email, Phone, Designation, PasswordSalt, PasswordHash
VEHICLE_CATEGORY	CategoryID (PK), CategoryName, Description
VEHICLE	VehicleID (PK), RegistrationNo, Make, Model, CategoryID (FK), OperationalStatus, BodyType, FuelType, Transmission, Seats
PRICING_RULE	PricingRuleID (PK), CategoryID (FK), MinDays, MaxDays, DailyRate, DiscountPercent, EffectiveFrom, EffectiveTo
BOOKING	BookingID (PK), CustomerID (FK), VehicleID (FK), PricingRuleID (FK), HandledBy (FK), StartDate, EndDate, BookingStatus, AgreedDailyRate, AppliedDiscountPercent, TotalAmount, CreatedAt
PAYMENT	PaymentID (PK), BookingID (FK), Amount, PaymentDate, PaymentMethod, PaymentStatus, TransactionRef
QUERY_LOG	LogID (PK), ExecutedAt, UserRole, UserEmail, QueryType, SqlText, RowsAffected, DurationMs, Status, ErrorMessage

All seven foreign keys use ON UPDATE CASCADE and ON DELETE RESTRICT, so rental and payment history cannot disappear through cascading deletes; vehicles are retired and bookings cancelled instead.

Foreign key	References	On update	On delete
VEHICLE.CategoryID	VEHICLE_CATEGORY(CategoryID)	CASCADE	RESTRICT
PRICING_RULE.CategoryID	VEHICLE_CATEGORY(CategoryID)	CASCADE	RESTRICT
BOOKING.CustomerID	CUSTOMER(CustomerID)	CASCADE	RESTRICT
BOOKING.VehicleID	VEHICLE(VehicleID)	CASCADE	RESTRICT
BOOKING.PricingRuleID	PRICING_RULE(PricingRuleID)	CASCADE	RESTRICT
BOOKING.HandledBy	EMPLOYEE(EmployeeID)	CASCADE	RESTRICT
PAYMENT.BookingID	BOOKING(BookingID)	CASCADE	RESTRICT

5. NORMALISATION

The starting point (UNF) is a single rental record holding customer, vehicle, category and pricing details, with all payments packed into one field. Decomposition removes the repetition:

Normal form	Requirement	How it is met
1NF	Atomic values, no repeating groups	Each payment is stored as its own row in PAYMENT.
2NF	No partial dependency on a key	Every relation has a single-attribute primary key.
3NF	No transitive dependency	Customer, employee, vehicle, category and pricing data each live in their own relation.

AgreedDailyRate and AppliedDiscountPercent in BOOKING are kept on purpose: they record the price agreed at booking time and depend on BookingID, not on the current pricing rule. BodyType in VEHICLE implies the category; this is a documented trade-off, and the application only offers body types that match the category.

6. DATABASE IMPLEMENTATION

The script vehicle_rental_system.sql creates the database vehicle_rental_db (MySQL 8.0.16 or later) with eight InnoDB tables, two stored procedures, five triggers, indexes and the sample data. Rules within one row are CHECK and UNIQUE constraints; rules that compare rows are triggers:

Object	Type	What it enforces
sp_validate_booking, trg_booking_bi / _bu	Procedure + triggers	Locks the vehicle row (SELECT … FOR UPDATE); rejects overlaps, non-operational vehicles and pricing rules that do not match.
chk_booking_formula	CHECK	TotalAmount = ROUND(DATEDIFF(EndDate, StartDate) × AgreedDailyRate × (1 − AppliedDiscountPercent / 100), 2)
sp_validate_pricing_rule, trg_pricing_rule_bi / _bu	Procedure + triggers	Pricing tiers of one category never overlap in days and dates.
trg_payment_bi	Trigger	Successful payments never exceed the booking total.
uq_… and chk_… constraints	UNIQUE, CHECK	Unique e-mail, phone, licence, registration; valid dates, statuses, methods and values.

Because the trigger locks the vehicle row inside the same transaction, a second request for the same vehicle waits for the first to finish and is then rejected if it overlaps, with the message “Booking rejected: vehicle already booked for an overlapping period.” An index on (VehicleID, StartDate, EndDate) supports this check and the availability search.

6.1 Sample Data

Table	Rows	Contents
VEHICLE_CATEGORY / PRICING_RULE	3 / 9	Car, Bike, Scooter; three duration tiers each (0%, 5%, 10% discount)
VEHICLE	53	8 body types; 50 available, 3 under maintenance
CUSTOMER / EMPLOYEE	35 / 2	Demo logins; one Manager and one Rental Operator
BOOKING	90	38 Completed, 30 Confirmed, 11 Pending, 11 Cancelled
PAYMENT	101	90 Success, 6 Failed, 3 Refunded, 2 Pending

6.2 Pricing Plan

Category	1–2 days	3–6 days (5% off)	7+ days (10% off)
Car	₹1,500	₹1,300	₹1,100
Bike	₹600	₹500	₹400
Scooter	₹450	₹380	₹300

All rules are effective from 1 January 2025 with no end date; the rates are illustrative values for testing.

7. SYSTEM IMPLEMENTATION

Layer	Technology	Role
User interface (vrs.ui)	Java Swing, Java2D	Customer and employee screens, dark / light theme; database work on a background thread
Services (vrs.service)	Java 17+	Business rules: Auth, Booking, Payment, Vehicle, Stats, Employee
Database access (vrs.db)	JDBC, MySQL Connector/J	One Db helper: PreparedStatements, transactions, query log

Layer	Technology	Role
Database	MySQL 9.7	Constraints, triggers and procedures, the final authority on integrity

Transactions: booking requests, payments and cancellations (with refund) run with auto-commit off and are committed or rolled back as one unit. **Security:** only SHA-256(salt + password) is stored, every statement uses PreparedStatement, which prevents SQL injection, and employees cannot sign themselves up. **Query log:** every statement is saved with its bound values, user, rows, time and result on a separate connection, so entries survive a rollback.

Availability search: vehicles that are operational and have no overlapping non-cancelled booking, priced by the matching rule:

```
SELECT v.VehicleID, v.Make, v.Model, v.BodyType, r.PricingRuleID, r.DailyRate, r.DiscountPercent
FROM VEHICLE v JOIN PRICING_RULE r ON r.CategoryID = v.CategoryID
  AND ? >= r.MinDays AND (r.MaxDays IS NULL OR ? <= r.MaxDays)
  AND ? >= r.EffectiveFrom AND (r.EffectiveTo IS NULL OR ? <= r.EffectiveTo)
WHERE v.OperationalStatus = 'Available'
  AND NOT EXISTS (SELECT 1 FROM BOOKING b
                  WHERE b.VehicleID = v.VehicleID AND b.BookingStatus <> 'Cancelled'
                    AND b.StartDate < ? AND b.EndDate > ?)
ORDER BY r.DailyRate;
```

Booking workflow: the customer requests (Pending, vehicle held) → an employee confirms (recorded as HandledBy) → the customer pays → the employee completes. The dashboard chart uses:

```
SELECT v.BodyType, COUNT(*) AS Rentals, SUM(b.TotalAmount) AS Revenue
FROM BOOKING b JOIN VEHICLE v ON v.VehicleID = b.VehicleID
WHERE b.BookingStatus IN ('Confirmed', 'Completed')
  AND b.StartDate >= ? AND b.StartDate < ?
GROUP BY v.BodyType;
```

8. USER INTERFACE

The interface is styled after modern rental websites, with a drawn symbol for each vehicle type instead of photographs and a light / dark mode switch. Screenshots use the sample data.

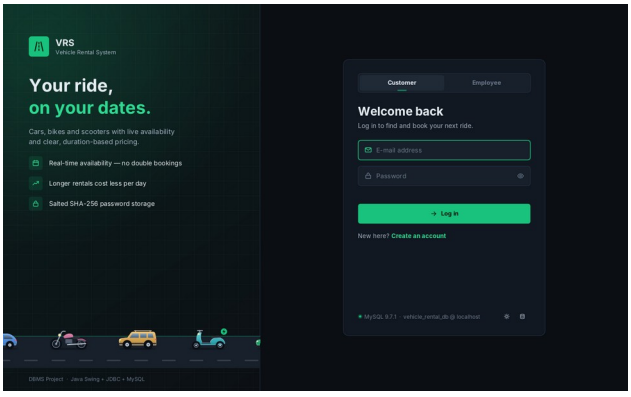


Figure 8.1: Login (customer / employee)

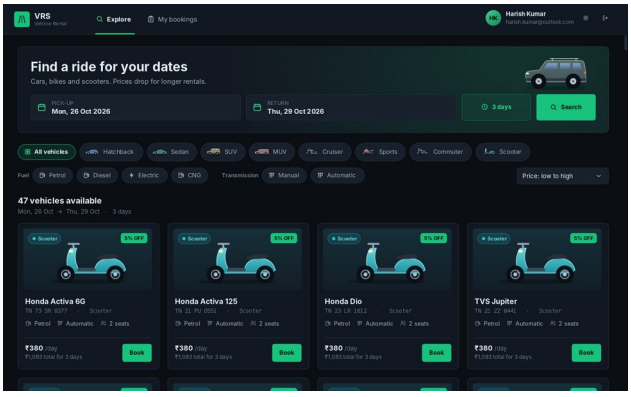


Figure 8.2: Customer – Explore

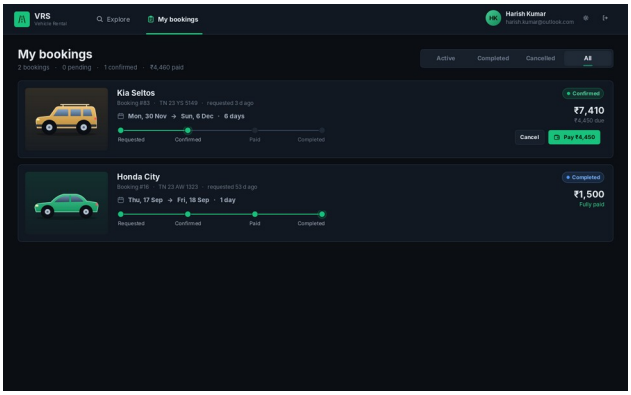


Figure 8.3: Customer – My bookings

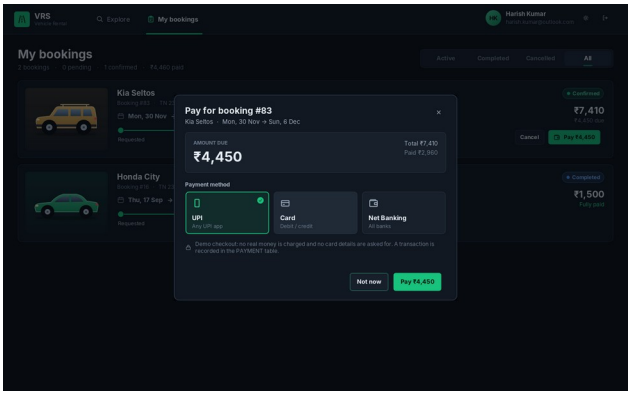


Figure 8.4: Payment after confirmation



Figure 8.5: Employee – Dashboard

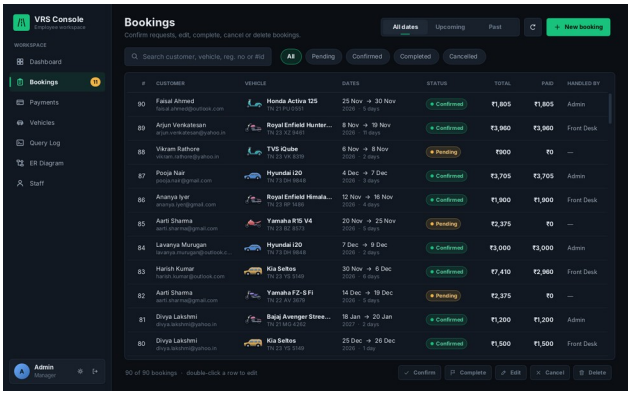


Figure 8.6: Employee – Bookings (CRUD)

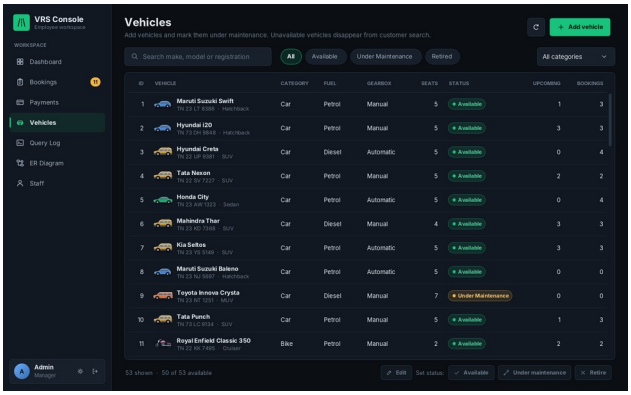


Figure 8.7: Vehicles and maintenance

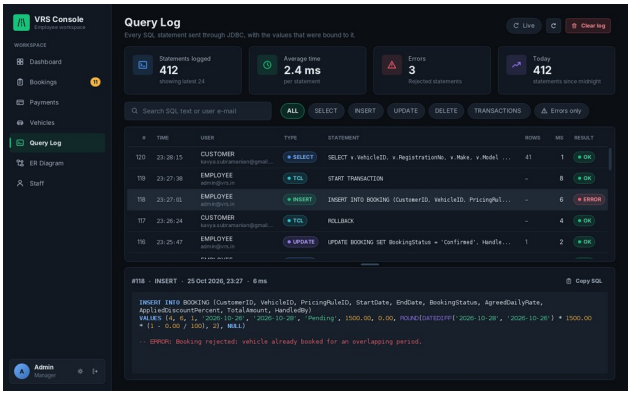


Figure 8.8: Query log

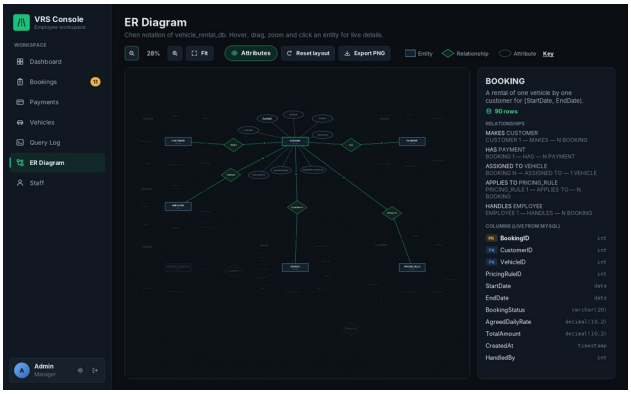


Figure 8.9: Interactive ER diagram

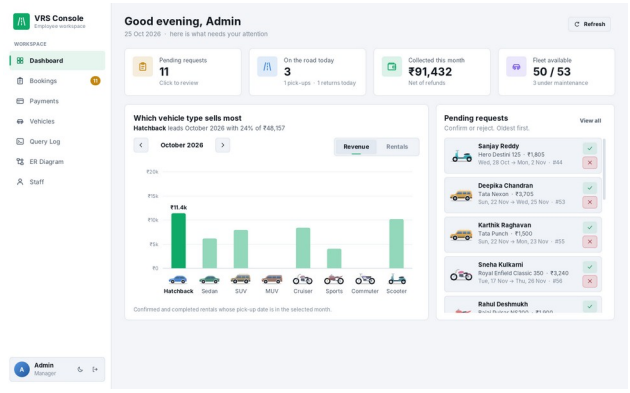


Figure 8.10: Light mode

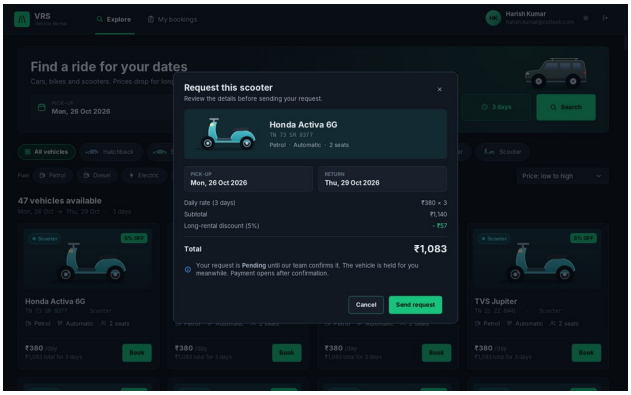


Figure 8.11: Booking request with price breakdown

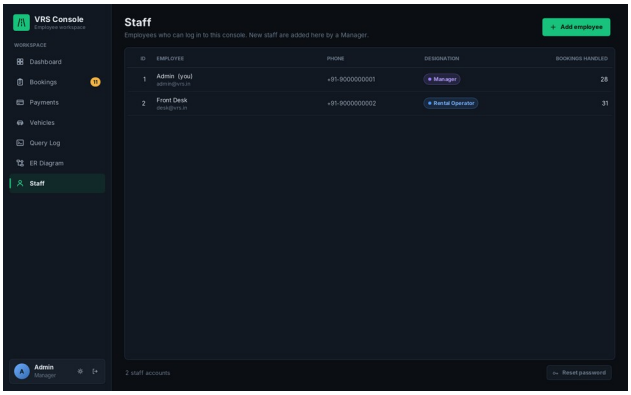


Figure 8.12: Staff management (Manager only)

9. TESTING

ID	Test	Expected result	Enforced by
TC-01	Log in with a wrong password	“Wrong password”	AuthService
TC-02	Rental Operator opens staff management	Page hidden; insert refused	EmployeeService
TC-03	Book vehicle 6 for 25–27 Oct 2026 (held by #47)	Rejected: overlap	trg_booking_bi
TC-04	Booking that starts on the day another ends	Accepted	Half-open interval
TC-05	Two requests for one vehicle at the same time	One succeeds, one rejected	FOR UPDATE lock
TC-06	Book a vehicle under maintenance	Rejected	sp_validate_booking
TC-07	4-day car booking	₹4,940	chk_booking_formula
TC-08	Customer pays a Pending booking	Refused until confirmed	PaymentService
TC-09	Payments above the booking total	Rejected	trg_payment_bi
TC-10	Delete a booking that has payments	Refused; cancel instead	FK RESTRICT
TC-11	Sign up with an e-mail that already exists	“Already registered”	uq_customer_email
TC-12	Return date not after pick-up date	Rejected	chk_booking_dates
TC-13	Cancel a paid booking	Cancelled; Refunded row added; vehicle free	BookingService transaction
TC-14	Add a pricing rule overlapping an existing tier	Rejected	trg_pricing_rule_bi
TC-15	Any action in the application	Statement, values, time and result in the log	Db + QueryLogger

10. CONCLUSION AND FUTURE WORK

The project delivers a working database application: seven entities and seven relationships map to eight normalised tables, and integrity is enforced in layers, with constraints for single rows, triggers and procedures across rows, and transactions with row locking for concurrent bookings. The Java Swing application gives customers and employees separate interfaces, and the query log and ER viewer make the database work visible.

Future work: a real payment gateway with e-mail / SMS alerts, pick-up locations, late fees and damage records, ratings and reviews, and a web or mobile client on the same database.

Important Design Decisions

- **Half-open rental intervals** allow back-to-back rentals without ambiguity.
- **Pending requests block the vehicle,** so two customers can never be promised the same dates.
- **Conflicts are checked in the database** with triggers and row locking, not only in the application.
- **Payment only after confirmation,** and bookings with payments are cancelled with a refund record rather than deleted.
- **Employees cannot self-register;** only a Manager creates staff accounts.
- **QUERY_LOG has no foreign keys** and is written on a separate connection, so the audit trail survives rollbacks.

References

1. Elmasri, R., and Navathe, S. B. Fundamentals of Database Systems, 7th ed. Pearson, 2016.
2. Silberschatz, A., Korth, H. F., and Sudarshan, S. Database System Concepts, 7th ed. McGraw Hill, 2019.
3. Oracle. MySQL Reference Manual and Connector/J Developer Guide. https://dev.mysql.com/doc/

Running the Application

Import sql/vehicle_rental_system.sql into MySQL, open the project in IntelliJ IDEA with the Connector/J jar added as a library, run Main.java and enter the MySQL password once. Demo accounts:

Role	E-mail	Password
Employee (Manager)	admin@vrs.in	Admin@123
Employee (Rental Operator)	desk@vrs.in	Staff@123
Customer	any sample customer, e.g. kavya.subramanian@gmail.com	Password@123